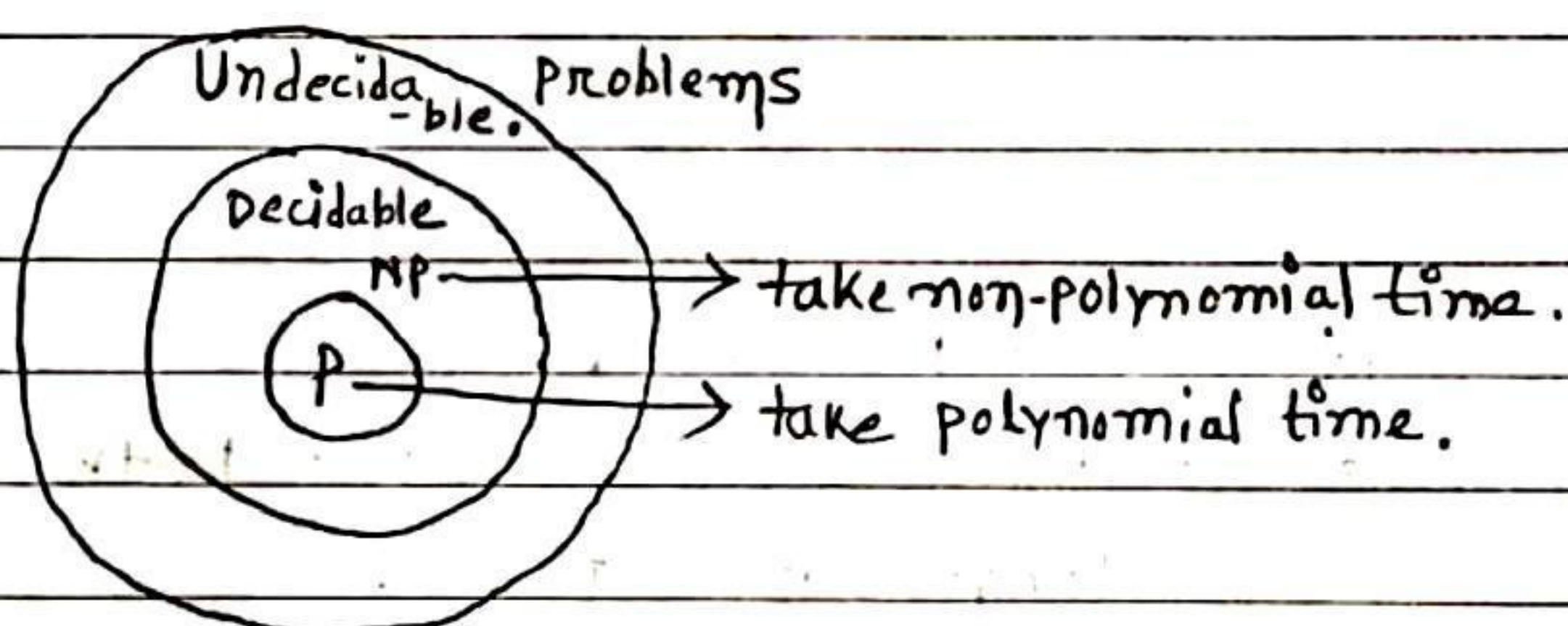
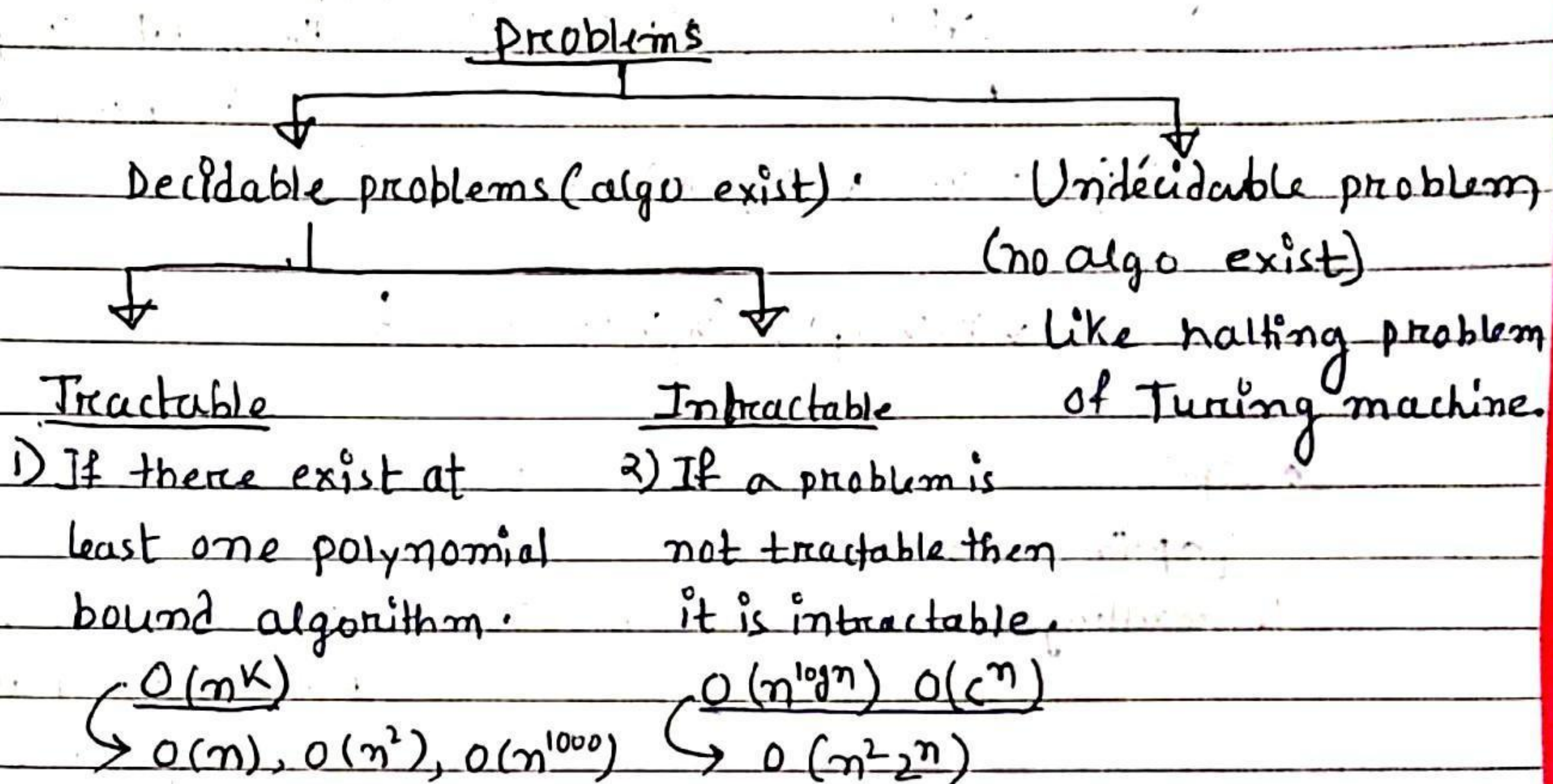


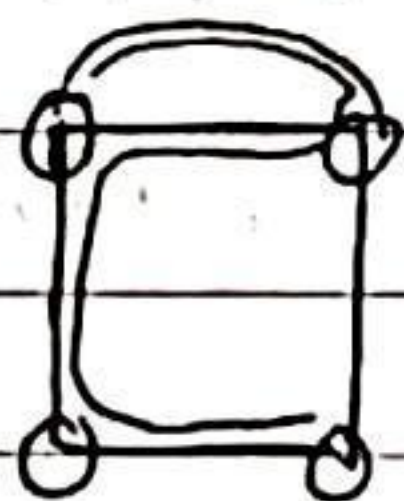
NP Completeness



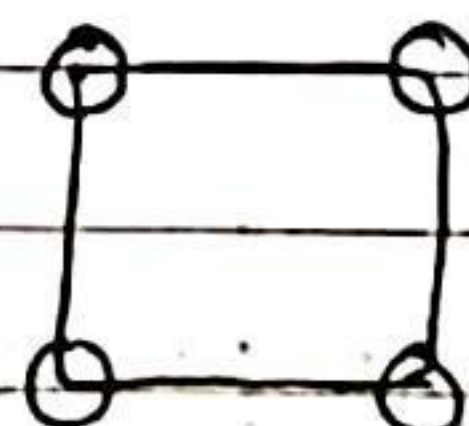
Some example = (look similar but time complexity is huge different)

① Shortest path problem (SP) $O(n^2 \log n)$ ——— Longest path problem (LP) $O(n!)$

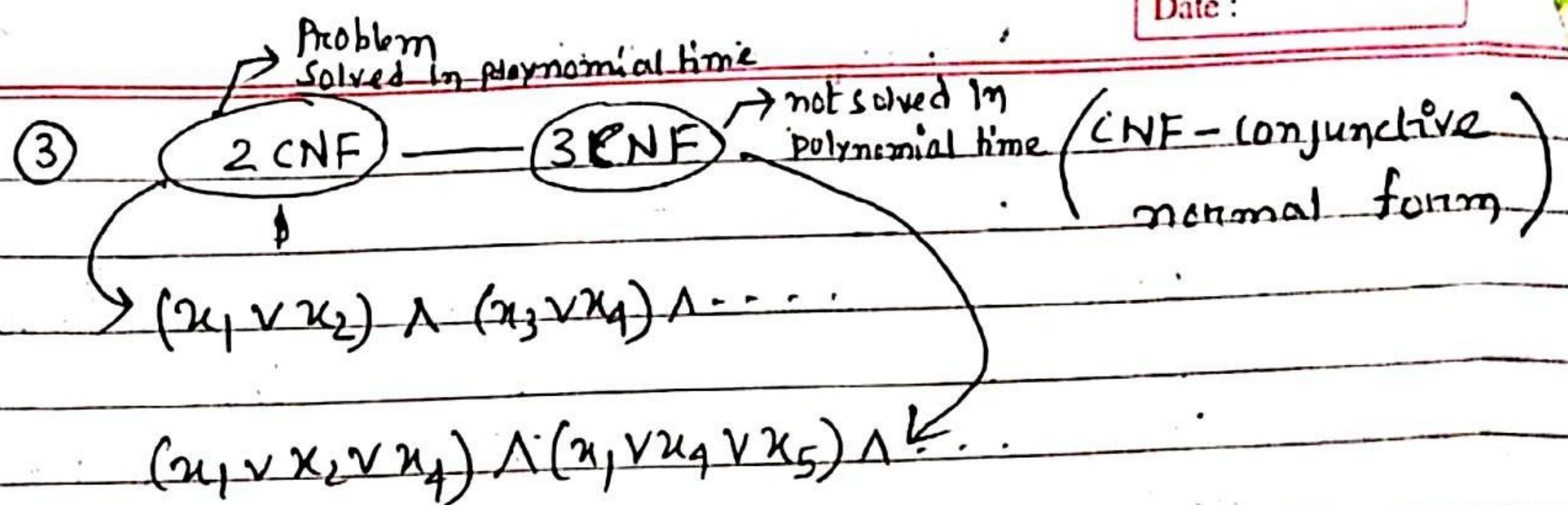
② ~~Euler tour~~ euler tour ——— hamiltonian cycle.



travel every edge ~~one~~ exactly once.
 $O(E)$
 $\rightarrow$ polynomial time



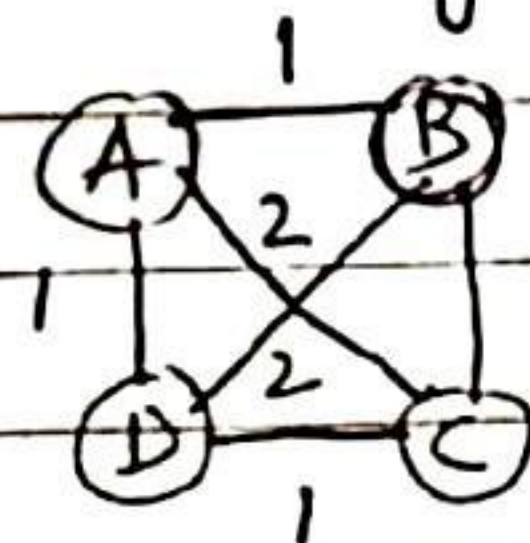
visit every vertex exactly once.
 $\rightarrow$ non-polynomial time



• Optimisation and decision problems -

Optimisation problem

1. Travelling salesman problem (TSP): A Graph G , shortest path covering all vertices exactly once.



$(A - B - C - D) - SP$

Decision problems:

1. Is there any shortest path covering all vertices of length at most 'K'.

— yes/no

Optimization problem

2. 0/1 Knapsack: Given P, W , Find out the max profit.

$$C=10, \quad P/W = \begin{matrix} I_1 & I_2 & I_3 \\ 10/5 & 5/2 & 6/6 \\ \checkmark & \checkmark & \end{matrix}$$

$$= \{I_1, I_2\} = \text{P} = 15$$

Decision problems

2. Is there any solution whose profit is atleast "K".

— yes/NO

3. ~~1~~ Optimisation problem -

3. Longest Common subsequence (LCS): A, B, find LCS.

$$\begin{matrix} A = aabb \\ B = abcd \end{matrix} \left. \vphantom{\begin{matrix} A = aabb \\ B = abcd \end{matrix}} \right\} \underline{LCS = ab}$$

Decision problem

3. Is there any subsequence whose length is atleast 'k'.

— (yes/no)

→ Every Optimisation Problem convert into decision problem.

point-1

→ If optimisation problem is easy ($O(n^k)$) then decision problem is easy.

point-2

→ If Decision problem is hard, then optimisation problem is hard.

• polynomial time reduction algorithm —

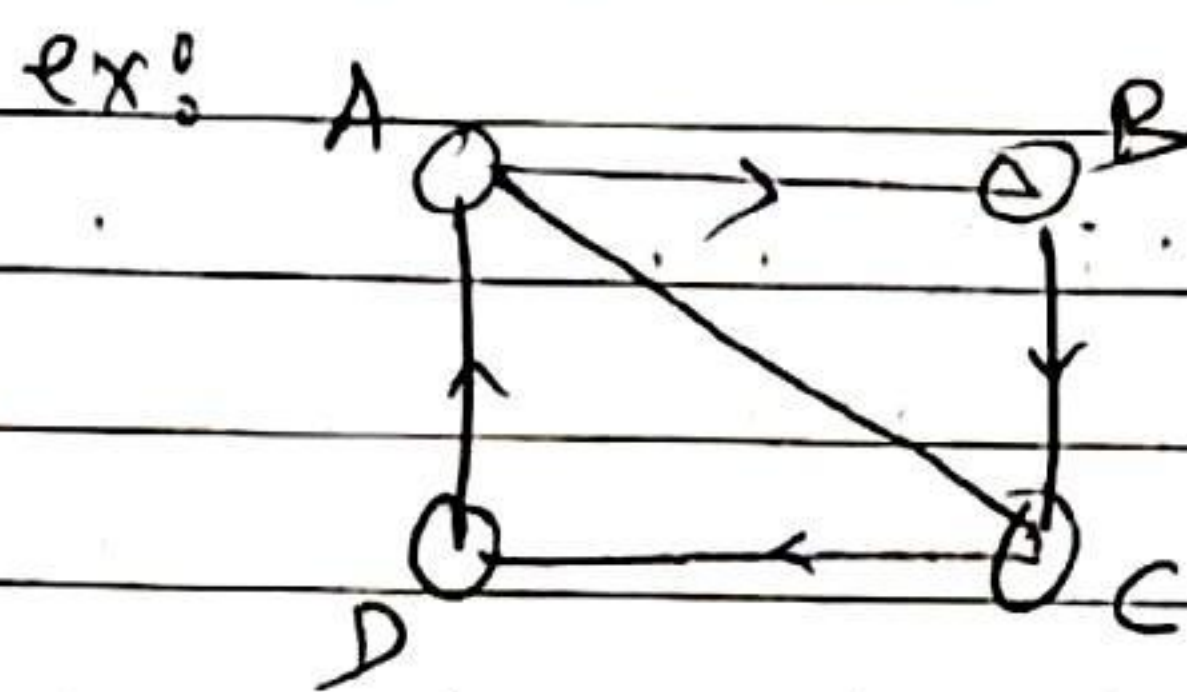


P → quickly solved or easily solved.
NP → quickly verified or easily verified.

→ A problem 'A' is said to be polynomial time reducible to a problem 'B' if

• Verification Algorithm $\Rightarrow$

$\rightarrow$ Verification Algorithm is given an solution you should be able to verify whether the solution right or not.



Is this Hamiltonian cycle?

Solution $\neq$ (A-B-C-D-A)

$\rightarrow$ YES.

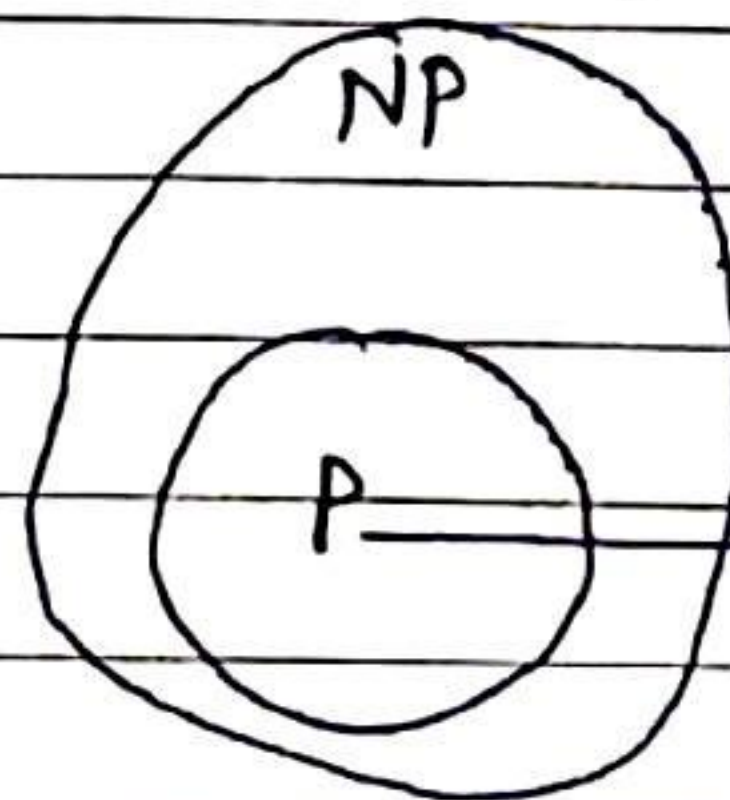
• P and NP Introduction -

~~P $\rightarrow$ set of all algorithm that can solve in polynomial time~~

~~NP $\rightarrow$ set of all decision problem for which there is a polynomial time verification~~

P $\rightarrow$ Set of all Decision problem, polynomial time algorithm to solve them.

NP $\rightarrow$ set of all Decision problem, for which there is a polynomial time verification.

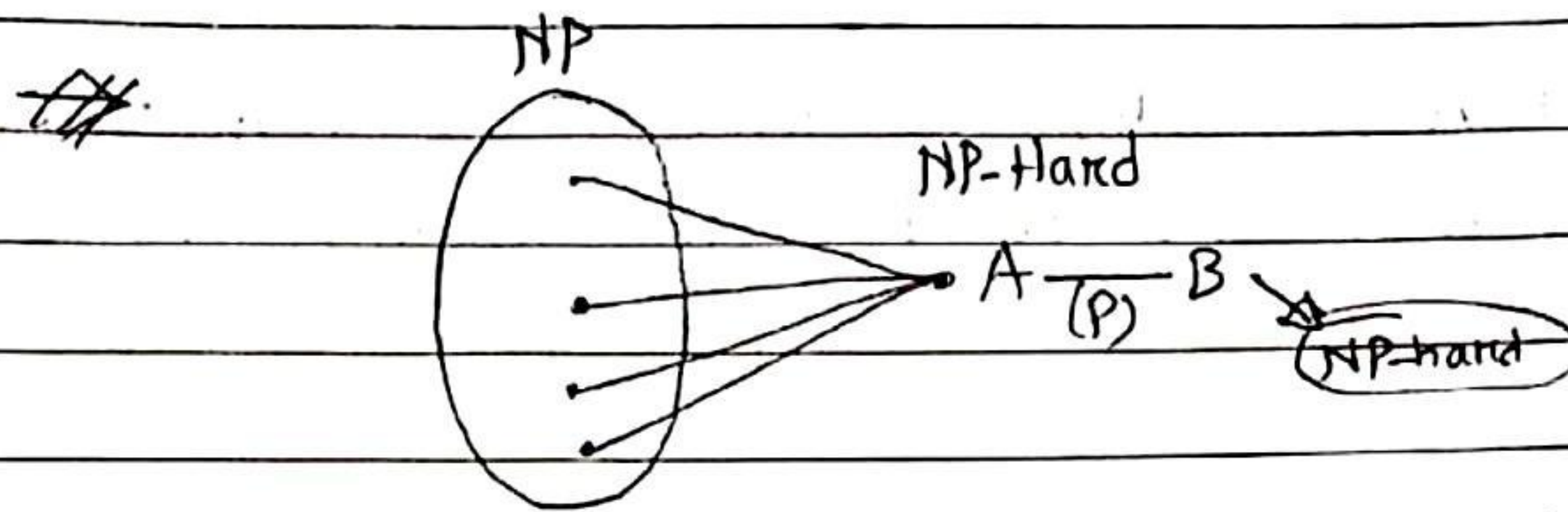


$\rightarrow$ solve in polynomial time.

$P \stackrel{?}{=} NP$

$P \subset NP$

• NP hard and NP Complete problems



→ If every problem in NP could be converted in a problem 'A' in polynomial time then we can say that the problem 'A' ~~can~~ ~~can't~~ if it could be solve in polynomial time, every ~~the~~ NP problem can be solved in polynomial time. (n^k)

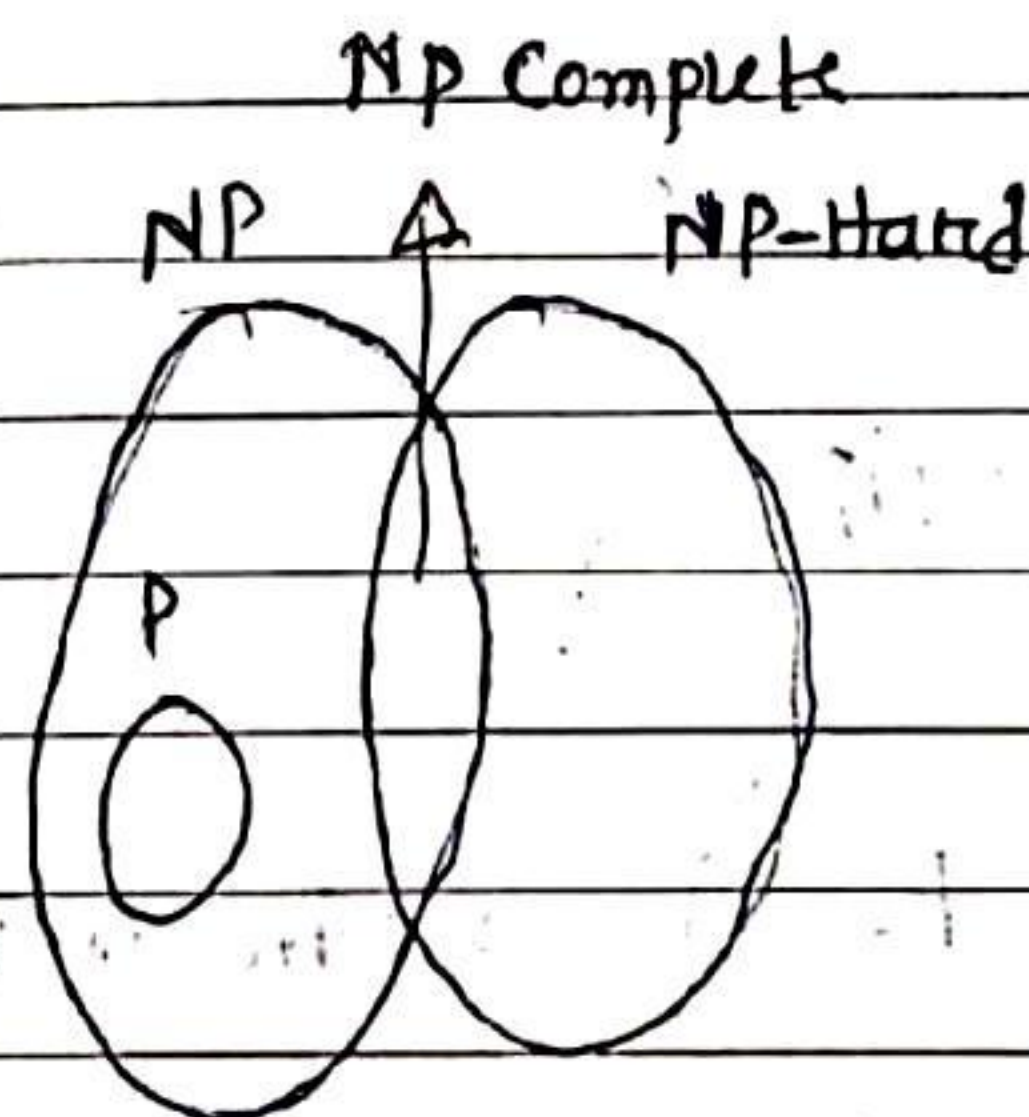
"NP=P"

NP Hard:

If every problem in NP can be polynomial time reducible to a problem 'A' then 'A' is called ~~as~~ NP-Hard.

NP-complete:

A problem said to be NP complete, if it is "NP" and "NP-Hard".



→ NP-Hard or NP-complete is solved in polynomial time, then $NP = P$.

→ NP problem or NP-complete is proven to be not solvable in polynomial time then, $P \neq NP$.

→ status of NP is unknown.

→ if A is NP-Hard, and A is converted in B in polynomial time, then B is NP-hard.

→ ~~convert~~ If A is NP-hard and B is NP, and A could be converted into B in polynomial time, then ^{we can say} B is NP complete.

• Well known NP complete problem →

